

任务10：完成 Qwen 的基础聊天功能

智能应用通常不是一开始就做复杂的多智能体和知识库问答，而是先完成“用户提问、后端调用大模型、返回回答”的最小聊天闭环。只有基础聊天能力稳定之后，后续才能继续扩展会话记忆、知识文档、RAG 检索增强以及更复杂的智能体工作流。Spring AI Alibaba 官方提供了 DashScope/Qwen 的快速接入方式，Spring AI 官方则提供了面向聊天模型的 `ChatClient` API。

本任务将实现 **Qwen 基础聊天功能**，下一次任务实现 **聊天会话记忆与历史上下文管理**。

1. 添加大模型依赖

在 `pom.xml` 中加入 Spring AI Alibaba 的 DashScope starter，用于接入 Qwen 模型。官方 DashScope 接入文档给出了通过 starter 配置模型的方式。

```
<!-- Spring AI Alibaba Agent Framework -->
<dependency>
    <groupId>com.alibaba.cloud.ai</groupId>
    <artifactId>spring-ai-alibaba-agent-framework</artifactId>
    <version>1.1.2.0</version>
</dependency>

<!-- DashScope ChatModel 支持 -->
<dependency>
    <groupId>com.alibaba.cloud.ai</groupId>
    <artifactId>spring-ai-alibaba-starter-dashscope</artifactId>
    <version>1.1.2.0</version>
</dependency>
```

2. 配置大模型访问参数

2.1 配置 DashScope API Key

在 `application.yml` 中增加大模型接入配置。建议通过环境变量读取 API Key。不要

在 `application.yml` 中增加配置，把密钥硬编码到公共仓库。官方 DashScope 接入文档支持通过配置方式接入模型。
临时生效：只对当前终端窗口有效；**永久生效：**写入系统环境变量或用户环境变量，之后新开的终端、IDEA 重启后都能读到

- 使用Windows PowerShell 永久设置示例：

```
# 用个人的百炼API Key代替YOUR_DASHSCOPE_API_KEY
[Environment]::SetEnvironmentVariable("AI_DASHSCOPE_API_KEY", "YOUR_DASHSCOPE_A
```

- Windows CMD 永久设置示例：

```
# 用个人的百炼API Key代替YOUR_DASHSCOPE_API_KEY
setx AI_DASHSCOPE_API_KEY "YOUR_DASHSCOPE_API_KEY"
```

2.2 修改 `application.yml`

在 `application.yml` 中对应增加，注意与之前设置的字段一致：

```
spring:
  ai:
    dashscope:
      api-key: ${AI_DASHSCOPE_API_KEY}
```

3. 新建后端聊天分层代码

3.1 新建请求对象

建议新建：

- `ChatRequestDTO`

`/model/dto/ChatRequestDTO.java`用于接收用户输入的问题文本。

```
import lombok.Data;
```

```
@Data
public class ChatRequestDTO {
    private String message;
}
```

3.2 新建返回对象

建议新建：

- ChatResponseVO

/model/vo/ChatResponseVO.java用于返回：

- 用户问题
- 大模型回答

```
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class ChatResponseVO {
    private String question;
    private String answer;
}
```

3.3 新建 Service 接口与实现类

建议新建：

- ChatService
- ChatServiceImpl

在 `Service` 中完成对 `ChatClient` 的调用，不要把大模型调用逻辑直接写在 `Controller` 中。Spring AI 官方推荐通过 `ChatClient` 统一发起聊天请求。

`ChatClient` 接口示例代码如下：

ChatService 接口示例代码如下：

```
public interface ChatService {  
    String chat(String message);  
}
```

ChatServiceImpl 实现类最简示例代码如下：

```
import com.alibaba.cloud.ai.dashscope.chat.DashScopeChatOptions;  
import com.example.demo.service.ChatService;  
import org.springframework.ai.chat.client.ChatClient;  
import org.springframework.stereotype.Service;  
  
@Service  
public class ChatServiceImpl implements ChatService {  
  
    private final ChatClient chatClient;  
  
    public ChatServiceImpl(ChatClient.Builder chatClientBuilder) {  
        this.chatClient = chatClientBuilder  
            .defaultSystem("你是一名专业、友好、简洁的中文智能助手，请根据用户  
            .defaultOptions(  
                DashScopeChatOptions.builder()  
                    .withTopP(0.7)  
                    .build()  
            )  
            .build();  
    }  
  
    @Override  
    public String chat(String message) {  
        return chatClient.prompt(message)  
            .call()  
            .content();  
    }  
}
```

3.4 新建聊天控制器

新建文件如下。

娃以莉娃。

- ChatController

实现提供基础聊天接口：

```
@RestController
@RequestMapping("/api/chat")
public class ChatController {

    private final ChatService chatService;

    public ChatController(ChatService chatService) {
        this.chatService = chatService;
    }

    @PostMapping
    public Result<ChatResponseVO> chat(@RequestBody ChatRequestDTO requestDTO)
    {
        String answer = chatService.chat(requestDTO.getMessage());
        ChatResponseVO responseVO = new ChatResponseVO(requestDTO.getMessage(),
        return Result.success(responseVO);
    }
}
```

附加题 1: 测试简单聊天

使用 Postman 调用聊天接口

```
{
  "message": "请用一句话介绍你自己"
}
```

若接口能成功返回模型回答，则说明基础聊天链路已经打通。

```
src/main/java
├─ controller
│   └─ ChatController.java
```

```
├─ service
│   ├── ChatService.java
│   └── impl
│       └── ChatServiceImpl.java
└─ model
    ├── dto
    │   └── ChatRequestDTO.java
    ├── vo
    │   └── ChatResponseVO.java
```

提交要求

1. 测试与任务成功示例:

截取聊天返回的测试结果图。

2. 文档提交: 将 更新后的项目核心代码推送至个人仓库, 并将截图粘贴至 Word 文档按时提交。